Big Data Assignment 2 Report

n.zagainov@innopolis.university
GITHUB: https://github.com/MatPlotLib-Guild/big-data-assignment2

Methodology

The search engine was implemented as a simple four-stage pipeline: service startup, data preparation, offline indexing, and online retrieval. HDFS is used to store input and index files, Hadoop Streaming MapReduce is used to build the index, Cassandra is used as the serving database, and Spark is used for data preparation and BM25 ranking. The main execution script is app.sh, which starts the services, creates the Python environment, installs the required packages, prepares the data, builds and stores the index, and finally runs a sample search query. The script start-services.sh starts HDFS, YARN, and the MapReduce history server, and also prepares a compact Spark archive in HDFS so that Spark-on-YARN submissions are stable.

The data preparation stage is implemented by prepare_data.py and prepare_data.sh. prepare_data.py reads one parquet file with Spark, selects the id, title, and text fields, samples a fixed number of rows, sanitizes file names, and writes each article as a plain UTF-8 text file named <doc_id>_<doc_title>.txt. prepare_data.sh uploads the parquet file to HDFS, runs prepare_data.py, recreates /input/data in HDFS, and uploads the generated text files there. Therefore the input to the indexer is a folder of plain text documents in HDFS.

The indexing stage is controlled by create_index.sh, index.sh, mapper1.py, reducer1.py, mapper2.py, reducer2.py, and split_index_output.py. index.sh is only a wrapper: it runs create_index.sh and then store_index.sh. create_index.sh executes two Hadoop Streaming pipelines. The first pipeline creates postings and document statistics. The second pipeline creates the vocabulary with document frequencies.

mapper1.py processes one input document at a time. It obtains the file path from the Hadoop Streaming environment, extracts doc_id and title from the file name, tokenizes the text, counts term frequencies inside the document, and counts the total number of tokens. Its output has two record types. For each distinct term it emits POSTING term doc_id title term_frequency. At the end of the document it emits DOC __meta__ doc_id title doc_length. reducer1.py aggregates records with the same key. For POSTING records it outputs POSTING term doc_id title total_term_frequency. For DOC records it outputs DOC doc_id title doc_length. While processing all DOC records, it also counts the number of documents and sums their lengths, and finally emits one corpus-level row CORPUS document_count total_doc_length. So pipeline 1 produces the raw material for the inverted index, document lengths, and corpus-wide statistics.

The mixed output of pipeline 1 is first stored in /tmp/indexer/pipeline1. Then split_index_output.py separates it into three local TSV files according to the record type: postings.tsv, doc_stats.tsv, and corpus_stats.tsv. These files are uploaded to the final HDFS index locations. /indexer/index stores the postings list, where each row represents one term in one document with its term frequency. /indexer/doc_stats stores per-document metadata, specifically document id, title, and document length. /indexer/corpus_stats stores corpus-level values, namely the number of indexed documents and the total document length, from which average document length is derived. These three artifacts are required by BM25.

The second MapReduce pipeline computes the vocabulary. mapper2.py reads the output of pipeline 1 and ignores everything except POSTING rows. For each posting it emits VOCAB term 1. Since each posting already corresponds to one term-document pair, counting these values gives the number of documents containing the term. reducer2.py sums the counts for each term and outputs VOCAB term document_frequency. The result is stored in /indexer/vocabulary. This artifact provides the document frequency needed for the IDF component of BM25.

The serving layer is implemented by store_index.sh and store_index.py. store_index.sh validates that all HDFS index artifacts exist, copies them into a local staging directory, waits until Cassandra is reachable, and then calls store_index.py. store_index.py creates the keyspace and four tables: postings_by_term, doc_stats, corpus_stats, and vocabulary. postings_by_term stores the inverted index keyed by term and doc_id. doc_stats stores title and document length per document. corpus_stats stores one global row with document_count, total_doc_length, and average_doc_length. vocabulary stores one row per term with its document frequency. Before loading, the tables are truncated so that the serving index is rebuilt from scratch in a consistent way.

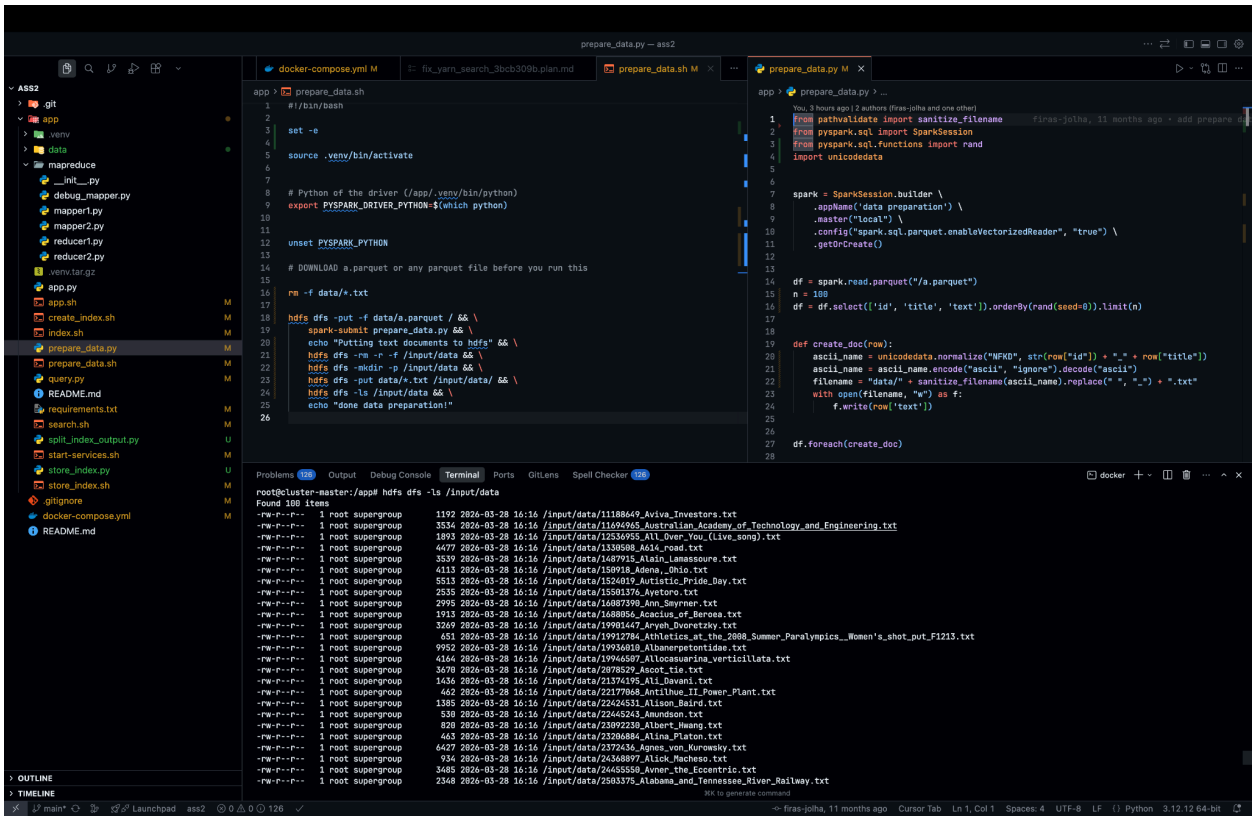
The ranker is implemented by query.py and search.sh. query.py reads the query text from the command line or stdin, tokenizes it, connects to Cassandra, loads corpus statistics, loads all document statistics, and retrieves postings and document frequencies only for the query terms. It then starts a Spark application, parallelizes the query term payloads, computes BM25 scores for each matching document, sums the partial scores across all query terms with reduceByKey, and returns the top 10 highest-scoring documents. The BM25 implementation uses document frequency, term frequency, document length, average document length, and corpus size, with fixed hyperparameters k1 = 1.5 and b = 0.75. search.sh is the user-facing command. It validates the query, activates the local virtual environment for the driver, submits query.py to Spark on YARN, and configures the executors to use the system Python interpreter available on the worker containers.

The required files in the assignment are therefore covered as follows. app.sh runs the full pipeline. start-services.sh starts the Hadoop services. prepare_data.py and prepare_data.sh create the input corpus. mapper1.py and reducer1.py implement the first indexing pipeline. mapper2.py and reducer2.py implement the second indexing pipeline. create_index.sh builds the HDFS index. store_index.sh and store_index.py load the index into Cassandra. index.sh runs the full indexing flow. query.py performs BM25 ranking. search.sh runs the query on YARN.

requirements.txt lists the Python dependencies. The final system keeps the architecture minimal: MapReduce is used only for offline index construction, Cassandra is used only as the serving store, and Spark is used only where distributed ranking is required.

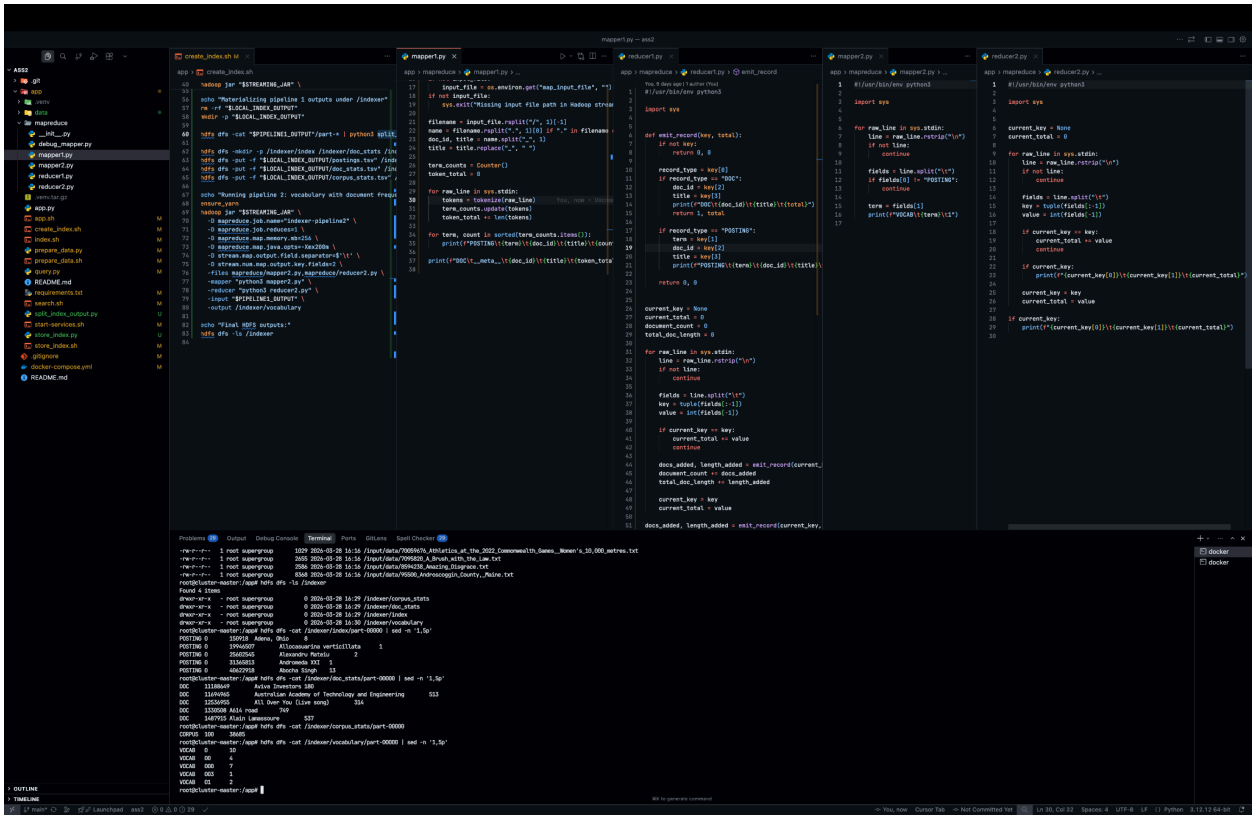
Demonstration

Proper work of prepare_data script: 100 documents (The assignment requires 100 documents for indexing in report instructions section and 1000 in description of the download pipeline. I am limited in resources hence I use 100 documents for putting into hdfs)

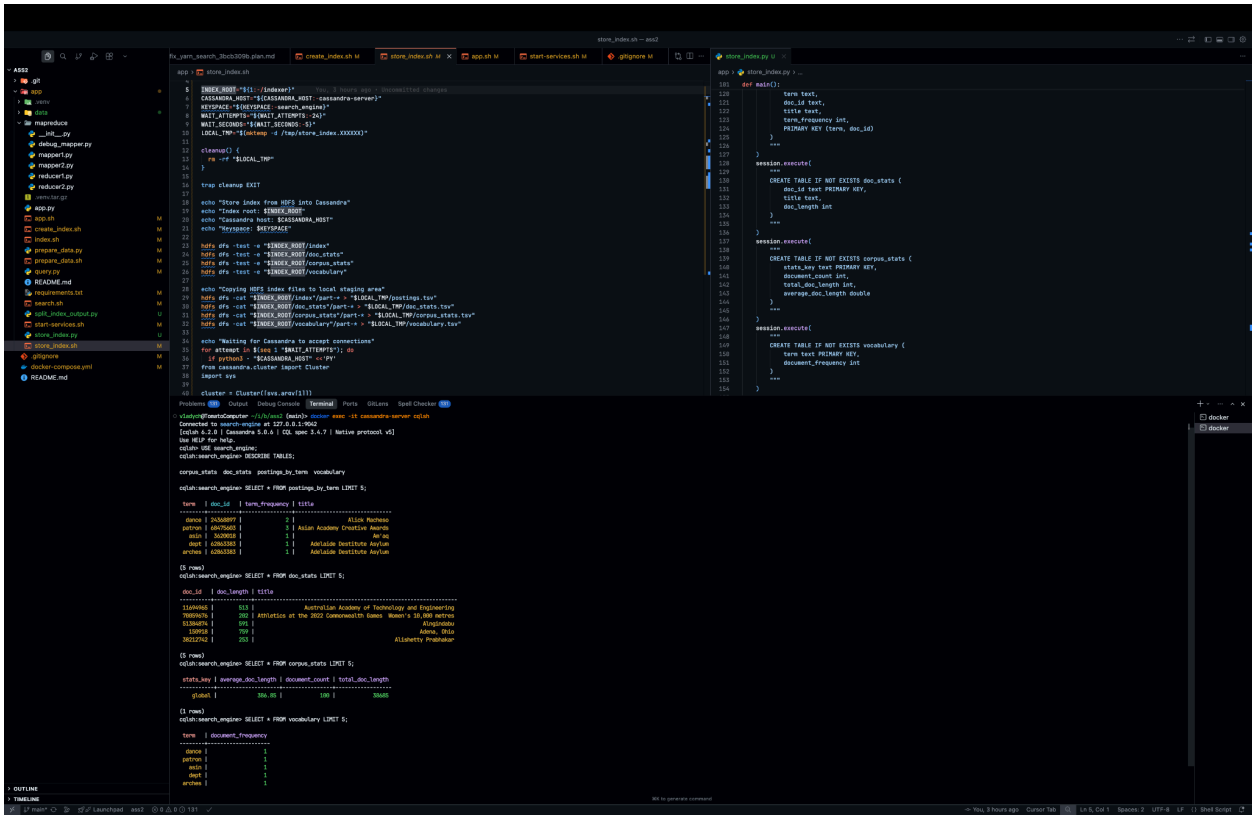


Index creation: successful indexing of 100 docks in HDFS and its transfer to cassandra

Mappers/Reducers scripts and results in HDFS

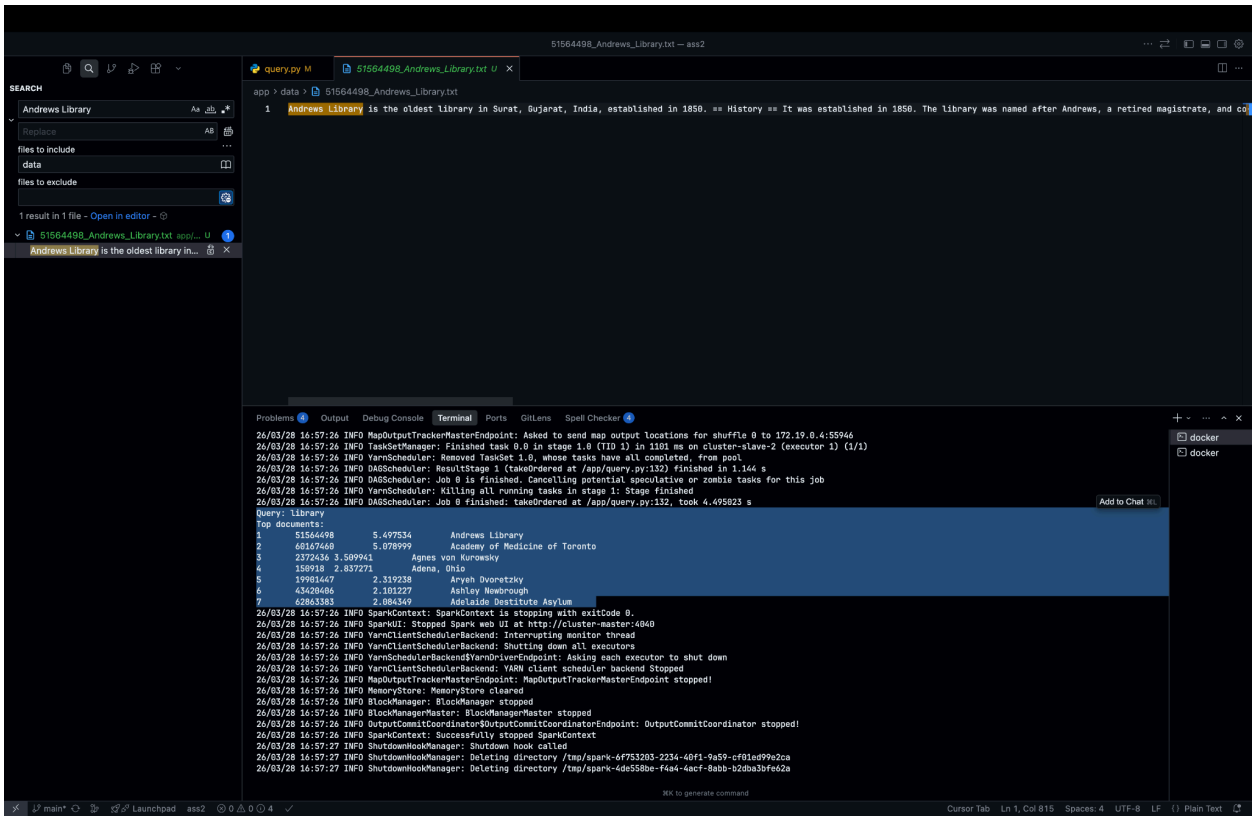


Cassandra transfer script and results in cassandra



Querying the index

Search results for query "library" and the contents of Top 1 doc



Search results for query "germany" and the contents of top 1 doc

5482275_Algirdas_Klimaitis.txt — ass2

SEARCH

Algirdas Klimaitis

Replace

files to include

data

files to exclude

1 result in 1 file - Open in editor -

5482275_Algirdas_Klimaitis.txt app/... U 1

Algirdas Klimaitis (1910 in KaunasBalti...

query.py M | 5482275_Algirdas_Klimaitis.txt U

app > data > 5482275_Algirdas_Klimaitis.txt

Germany 1 of 4

1 i Germany occupied Lithuania in June 1941, at the start of Operation Barbarossa, Klimaitis formed a military unit of roughly 600 members, which was not subordinate to the Lithuanian Activist Fr

Problems 19 | Output | Debug Console | Terminal | Ports | GitLens | Spell Checker 19

```
26/03/28 17:01:36 INFO BlockManagerInfo: Added broadcast_2_piece0 in memory on cluster-slave-1:38121 (size: 6.7 KiB, free: 366.3 MiB)
26/03/28 17:01:37 INFO MapOutputTrackerMasterEndpoint: Asked to send map output locations for shuffle 0 to 172.19.0.3:44420
26/03/28 17:01:37 INFO TaskSetManager: Finished task 0.0 in stage 1.0 (TID 1) in 1022 ms on cluster-slave-1 (executor 1) (1/1)
26/03/28 17:01:37 INFO YarnScheduler: Removed TaskSet 1.0, whose tasks have all completed, from pool
26/03/28 17:01:37 INFO DAGScheduler: ResultStage 1 (takeOrdered at /app/query.py:132) finished in 1.124 s
26/03/28 17:01:37 INFO DAGScheduler: Job 0 is finished. Cancelling potential speculative or zombie tasks for this job
26/03/28 17:01:37 INFO YarnScheduler: Killing all running tasks in stage 1: Stage finished
26/03/28 17:01:37 INFO DAGScheduler: Job 0 finished: takeOrdered at /app/query.py:132, took 4.235286 s
Query: germany
Top documents:
1       5482275 5.945171        Algirdas Klimaitis
2       3483810 4.944996        Albert Beier
3       52696556        4.466242        Andrea Fenzau-Lehmann
4       16087390        4.276798        Ann Smyrner
26/03/28 17:01:37 INFO SparkContext: SparkContext is stopping with exitCode 0.
26/03/28 17:01:37 INFO SparkUI: Stopped Spark web UI at http://cluster-master:4040
26/03/28 17:01:37 INFO YarnClientSchedulerBackend: Interrupting monitor thread
26/03/28 17:01:37 INFO YarnClientSchedulerBackend: Shutting down all executors
26/03/28 17:01:37 INFO YarnSchedulerBackend$YarnDriverEndpoint: Asking each executor to shut down
26/03/28 17:01:37 INFO YarnClientSchedulerBackend: YARN client scheduler backend Stopped
26/03/28 17:01:37 INFO MapOutputTrackerMasterEndpoint: MapOutputTrackerMasterEndpoint stopped!
26/03/28 17:01:37 INFO MemoryStore: MemoryStore cleared
26/03/28 17:01:37 INFO BlockManager: BlockManager stopped
26/03/28 17:01:37 INFO BlockManagerMaster: BlockManagerMaster stopped
26/03/28 17:01:37 INFO OutputCommitCoordinator$OutputCommitCoordinatorEndpoint: OutputCommitCoordinator stopped!
26/03/28 17:01:37 INFO SparkContext: Successfully stopped SparkContext
26/03/28 17:01:38 INFO ShutdownHookManager: Shutdown hook called
26/03/28 17:01:38 INFO ShutdownHookManager: Deleting directory /tmp/spark-d6d6ffa5-4e5d-4ff6-83ea-742f9a880be8
26/03/28 17:01:38 INFO ShutdownHookManager: Deleting directory /tmp/spark-4da5f892-a96f-438f-bd75-fc9a7cba3e5f/pyspark-d5eb57b6-67e3-4cdf-9b4e-d257f9672d37
26/03/28 17:01:38 INFO ShutdownHookManager: Deleting directory /tmp/spark-4da5f892-a96f-438f-bd75-fc9a7cba3e5f
root@cluster-master:/app#
```

Add to Chat

docker

docker

⌘K to generate command

main* | Launchpad | ass2 | 0 0 19 | Cursor Tab | Ln 1, Col 423 (7 selected) | Spaces: 4 | UTF-8 | LF | Plain Text